

Física discreta verificada por SAT:

Un flujo de trabajo SAT mínimo y auditable para geometría discreta finita y localidad

Oscar Riveros

Polímata, Chile

independent.academia.edu/oarr • github.com/maxtuno

2025-12-29

Resumen

Presentamos un *artículo de métodos*: un flujo de trabajo pequeño y auditable que compila *restricciones de física discreta finita* a instancias de SAT con testigos verificables mecánicamente. El objetivo *no* es reclamar una unificación en el continuo, sino ofrecer una vía reproducible para (i) **verificar** la satisfacibilidad de restricciones de modelos discretos, o (ii) **refutarlas** (UNSAT) dentro de regímenes finitos explícitos.

Se entregan dos familias de ejemplos completamente chequeables:

1. **Gravedad 2D combinatoria (equilátera)**: codificamos la existencia de un 2-complejo simplicial cerrado mediante variables de incidencia de triángulos, imponemos restricciones de valencia por vértice y validamos la suma de curvatura discreta de Gauss–Bonnet. Incluimos casos SAT (esfera octaédrica) y UNSAT (intento de “esfera plana”).
2. **Dinámica booleana de localidad (influencia a velocidad finita)**: un modelo discreto en el tiempo de propagación de influencia en grafos que impone un cono de luz exacto (análogo booleano de la localidad tipo Lieb–Robinson). Proveemos instancias SAT/UNSAT de alcanzabilidad parametrizadas por tamaño del grafo y horizonte temporal.

Reportamos el crecimiento de cláusulas/variables bajo una reducción one-hot e incluimos tiempos base de resolución usando un prototipo CDCL en Python, enfatizando que el entregable principal es la *codificación auditable + validador*, compatible con solvers SAT industriales.

Palabras clave. SAT; geometría discreta; cálculo de Regge; Gauss–Bonnet; cotas de Lieb–Robinson; compilación de restricciones; verificación reproducible; ruptura de simetría; codificaciones de cardinalidad; GCNF.

1. Alcance y afirmaciones (estatus epistémico)

Este trabajo afirma *solo* lo siguiente:

- Un **workflow de compilación en dominio finito** desde restricciones de modelos discretos a SAT/MaxSAT.
- Un **formato intermedio mínimo (GCNF)** fácil de auditar.
- Un **toolchain de verificación** que valida testigos respecto de las restricciones originales.

- **Instancias toy** no triviales en el sentido de incluir casos SAT y UNSAT, y al menos una instancia hallada por búsqueda (no hard-coded).

De forma explícita *no* afirmamos: (i) derivación de GR/QFT continuas desde estos toys, (ii) consecuencias observacionales en cosmología, ni (iii) que las instancias presentadas sean duras a escala de investigación.

1.1. Contexto dentro de un programa más amplio de física discreta

El workflow es compatible con programas discretos más amplios (p.ej. gravedad simplicial, triangulaciones causales, restricciones operacionales de localidad y pipelines de “aritmética exacta” orientados a constraints). En particular, puede servir como sustrato de verificación para cualquier marco que trate afirmaciones físicas como *conjuntos finitos refutables de restricciones* en lugar de axiomas en el continuo. Este artículo permanece estrictamente en la capa “métodos + instancias finitas”.

2. Trabajo relacionado (posicionamiento)

El **cálculo de Regge** introdujo discretizaciones piecewise-linear de GR sobre complejos simpliciales [1]. Trabajo numérico y estructural en Regge y gravedad discreta entrega contexto sobre elecciones de discretización y problemas de gauge [6, 7].

La **gravedad cuántica basada en triangulaciones** (incluyendo triangulaciones dinámicas causales) es un campo maduro; véase la revisión en Physics Reports [5].

Las **cotas de Lieb–Robinson** establecen una velocidad efectiva finita para propagación de información en sistemas cuánticos locales en retículas [2, 3]. Nuestra “dinámica booleana de localidad” no es una cota en norma de operadores, sino una familia deliberadamente simplificada de restricciones de propagación a velocidad finita.

En SAT, las reducciones de CSP en dominios finitos a SAT y el diseño de codificaciones SAT para restricciones globales (en especial **cardinalidad**) son bien conocidas. Usamos una codificación tipo sequential counter para restricciones exact- k [4]. Como trasfondo opcional sobre codificaciones y comparaciones, véase [8, 9].

Nuestro aporte no es un nuevo modelo de gravedad ni un nuevo teorema de localidad; es el **punto auditable**:

$$\text{restricciones discretas finitas} \rightarrow \text{instancia SAT} \rightarrow \text{testigo} \rightarrow \text{validación mecánica}.$$

3. Método: desde restricciones discretas a SAT

3.1. GCNF (CNF generalizada sobre igualdades)

Usamos un lenguaje de cláusulas sobre variables de dominio finito:

- Variables: $x_i \in \{0, 1, \dots, b_i - 1\}$.
- Literal atómico: $x_i = a$.
- Cláusula: disyunción $(x_{i_1} = a_1) \vee \dots \vee (x_{i_m} = a_m)$.
- Fórmula: conjunción de cláusulas.

Un archivo **GCNF** es una lista orientada por líneas de listas Python:

Listing 1: Formato de una línea-cláusula en GCNF.

```
[(var_id, value), (var_id, value), ...]
```

Cada línea es una cláusula. La representación es deliberadamente simple: es fácil de auditar por inspección y fácil de traducir a CNF estándar.

3.2. Reducción one-hot a CNF

Para cada variable multi-valuada x con tamaño de dominio b , introducimos indicadores booleanos $X_{x,0}, \dots, X_{x,b-1}$ con restricciones **exactly-one**. Entonces:

- El literal $(x = a)$ se convierte en $X_{x,a}$.
- Una cláusula GCNF se convierte en una cláusula CNF estándar sobre estos indicadores.

La herramienta `gcnf_to_cnf.py` produce un DIMACS CNF y un archivo mapa para traducir entre testigos en el espacio CNF y asignaciones del dominio original.

3.3. Bucle de verificación (chequeo del testigo)

Dado un testigo de un solver SAT, lo verificamos **contra las restricciones originales GCNF**, no solo contra el CNF reducido. El validador (`validate_solution.py`) chequea cada cláusula del GCNF bajo la asignación decodificada. Esto separa “corrección del solver” de “corrección del encoding” y hace el pipeline auditable.

3.4. Ruptura de simetría como fijación de gauge discreta

Los modelos discretos por restricciones suelen tener grandes grupos de automorfismos (p.ej. relabeling de vértices). Agregar **restricciones de ruptura de simetría** puede acelerar la búsqueda, pero debe tratarse como *elección de gauge* (elimina soluciones equivalentes, no “física”). Usamos ruptura opcional en algunas instancias y discutimos el riesgo de sobre-restringir (§7).

4. Toy-GR-Complex: síntesis de una 2-variedad equilátera y chequeo de Gauss–Bonnet

4.1. Curvatura combinatoria

Para una triangulación **equilátera**, cada triángulo incidente contribuye ángulo $\pi/3$ en un vértice, por lo tanto

$$K(v) = 2\pi - \deg(v) \frac{\pi}{3}, \quad \kappa(v) := 6 - \deg(v),$$

y la identidad discreta de Gauss–Bonnet es

$$\sum_v \kappa(v) = 6\chi.$$

Para una 2-esfera triangulada ($\chi = 2$) se obtiene $\sum_v \kappa(v) = 12$.

4.2. Codificando la existencia de un 2-complejo simplicial cerrado

Sea $V = \{1, \dots, N\}$. Para cada triple $i < j < k$, introducimos una variable booleana

$$\tau_{ijk} \in \{0, 1\}$$

que indica si el triángulo (i, j, k) está presente.

Restricción de variedad en aristas (sin borde). Para cada arista no ordenada $\{i, j\}$, consideramos $S_{ij} = \{\tau_{ijk} : k \neq i, j\}$. Forzamos que el número de triángulos incidentes sea **0 o 2**, esto es: (i) “a lo más 2” (prohibir cualquier triple simultáneamente verdadero), y (ii) “no exactamente 1” (si uno es verdadero, al menos otro debe serlo).

Restricción de valencia en vértices. Definimos el grado del vértice como la cantidad de triángulos incidentes:

$$\deg(v) = \sum_{i < j, i, j \neq v} \tau_{vij}.$$

Codificamos $\deg(v) = d_v$ con un encoding exact- k por sequential counter [4].

4.3. Ejemplo SAT: $N = 6$, $\deg(v) = 4$ (esfera octaédrica)

Objetivo: $\deg(v) = 4$ para todo v . Entonces

$$\sum_v \kappa(v) = \sum_v (6 - 4) = 12,$$

consistente con una 2-esfera.

Un testigo SAT corresponde al octaedro (8 triángulos):

- (1, 2, 3)
- (1, 2, 6)
- (1, 3, 5)
- (1, 5, 6)
- (2, 3, 4)
- (2, 4, 6)
- (3, 4, 5)
- (4, 5, 6)

4.4. Ejemplo UNSAT: $N = 6$, $\deg(v) = 6$ (intento de “esfera plana”)

Imponer $\deg(v) = 6$ para todo vértice fuerza $\sum_v \kappa(v) = 0$, i.e. $\chi = 0$ (presupuesto de curvatura tipo toro), lo que contradice las mismas restricciones de variedad cerrada para $N = 6$ en este encoding. SAT retorna **UNSAT**, entregando una refutación concreta en el régimen finito.

4.5. Ejemplo SAT hallado por búsqueda: $N = 7$, patrón $(5, 5, 4, 4, 4, 4, 4)$

Aquí $\sum_v \deg(v) = 30$, luego $F = \frac{1}{3} \sum_v \deg(v) = 10$, y $\sum_v \kappa(v) = 6N - \sum_v \deg(v) = 42 - 30 = 12$. SAT encuentra una esfera triangulada con 10 triángulos:

- $(1, 3, 6)$
- $(1, 3, 7)$
- $(1, 4, 5)$
- $(1, 4, 7)$
- $(1, 5, 6)$
- $(2, 3, 6)$
- $(2, 3, 7)$
- $(2, 4, 5)$
- $(2, 4, 7)$
- $(2, 5, 6)$

Este ejemplo no está hard-coded: el solver busca en un espacio candidato de 35 triángulos y encuentra una 2-variedad consistente que satisface las restricciones de grado.

5. Toy-LR-Dynamics: influencia booleana a velocidad finita en grafos

5.1. Modelo discreto de influencia

Sea $G = (V, E)$ un grafo. Definimos variables booleanas $I(v, t) \in \{0, 1\}$ que indican si el sitio v está “influenciado” al tiempo t . Fijamos un horizonte T .

Para cada vértice v y paso $t \rightarrow t + 1$, imponemos la actualización local exacta:

$$I(v, t + 1) \iff \left(I(v, t) \vee \bigvee_{u \in N(v)} I(u, t) \right).$$

Esto impone un **cono de luz** estricto: la influencia se propaga a velocidad 1 arista por paso, por lo que

$$I(v, T) = 1 \Rightarrow d(v, v_0) \leq T$$

para un único sitio inicial v_0 . Esto debe leerse como una familia finita y determinista de restricciones de localidad inspirada en el contenido cualitativo de Lieb–Robinson [2, 3].

5.2. Instancias SAT/UNSAT de alcanzabilidad en caminos

En un grafo camino de largo L con sitio inicial 1 influenciado en $t = 0$, preguntamos si el sitio L puede ser influenciado a tiempo T :

- Si $T < L - 1$: debe ser **UNSAT** (fuera del cono de luz).
- Si $T = L - 1$: es **SAT** (frontera del cono).

Estas instancias entregan un “harness” de refutación limpio: el solver no puede “hacer trampa” asignando valores futuros arbitrarios porque imponemos la implicación inversa en la bicondicional.

6. Benchmarks: crecimiento de tamaño y tiempos base

6.1. Tabla resumen (instancias representativas)

La Tabla 1 reporta: número de cláusulas GCNF (líneas), tamaño CNF tras reducción one-hot (variables/cláusulas), y tiempo base de resolución usando el prototipo CDCL en Python.

Nota: el solver prototipo no está optimizado; los conteos de cláusulas/variables son la métrica estable. Cualquier solver CDCL compatible con DIMACS puede sustituirse.

Instancia	Resultado	Solve_s	GCNF_clauses	CNF_vars	CNF_clauses
Camino LR $L=8, T=3$	UNSAT	0.134	99	64	163
Camino LR $L=8, T=7$	SAT	0.231	219	128	347
Triangulación $N=6$, $\deg=4$	SAT	0.240	1441	1360	2801
Triangulación $N=6$, $\deg=6$	UNSAT	0.270	1441	1360	2801
Triangulación $N=7$, $(5, 5, 4, 4, 4, 4, 4)$	SAT	4.377	3675	3430	7105

Cuadro 1: Instancias representativas: tamaños y tiempos base.

Una instancia de triangulación más grande ilustra el blow-up rápido en problemas de búsqueda:

Instancia	Resultado	Solve_s	GCNF_clauses	CNF_vars	CNF_clauses
Triangulación $N=8$, $(5 \times 4, 4 \times 4)$	TIMEOUT	50	8120	7504	15624

Cuadro 2: Instancia mayor: timeout en el prototipo (corte 50s).

6.2. Escalamiento de localidad en caminos (benchmark por familia)

L	T	Resultado	Solve_s	GCNF_clauses	CNF_vars	CNF_clauses
12	3	UNSAT	0.128	151	96	247
16	4	UNSAT	0.201	265	160	425
20	5	UNSAT	0.265	411	240	651
24	6	UNSAT	0.187	589	336	925

Cuadro 3: Benchmark por familia: alcanzabilidad UNSAT en caminos.

Alcanzabilidad UNSAT (fuera del cono).

Alcanzabilidad SAT (frontera del cono).

6.3. Métrica simple de “journal-readiness” para instancias

Para hacer evaluable la “no-trivialidad” y la escalabilidad, recomendamos reportar

$$\mathcal{M} = (V_{\text{máx}}, b_{\text{máx}}, C_{\text{máx}}, T_{\text{solve}}, \epsilon),$$

L	T	Resultado	Solve_s	GCNF_clauses	CNF_vars	CNF_clauses
12	11	SAT	0.155	519	288	807
16	15	SAT	0.161	947	512	1459
20	19	SAT	0.173	1503	800	2303
24	23	SAT	0.249	2187	1152	3339

Cuadro 4: Benchmark por familia: alcanzabilidad SAT en la frontera.

donde $V_{\text{máx}}$ es el mayor tamaño de vértices/grafos resuelto bajo un presupuesto de tiempo fijo, $b_{\text{máx}}$ es el mayor tamaño de paleta/dominio usado en encodings métricos, $C_{\text{máx}}$ es el mayor número de cláusulas CNF manejado, T_{solve} es una estadística representativa del tiempo de resolución, y ϵ es un error físico significativo (p.ej. error de curvatura bajo discretización por paleta o slack bajo MaxSAT). Este artículo reporta $V_{\text{máx}}$, $C_{\text{máx}}$ y T_{solve} para las familias toy con dominios booleanos ($b = 2$); reportar ϵ se vuelve significativo una vez que se agreguen encodings de paleta métrica (no booleanos, $b > 2$).

7. Gauge, redundancia y qué significa “difeomorfismo” aquí

7.1. Gauge de simetría (etiquetas) en encodings combinatorios

El modelo de síntesis de triangulaciones es invariante bajo relabeling de vértices. Esto es **gauge puro** en el encoding. La ruptura parcial (fijar un triángulo, fijar identidad de un vértice de grado 5, etc.) puede acelerar SAT pero también puede **sobre-restringir** si se aplica de forma inconsistente (observado en la práctica en instancias $N = 7$).

7.2. Gauge de Regge vs. lo que codificamos

En Regge completo, las longitudes de arista cargan redundancias análogas a libertad difeomórfica; pequeños desplazamientos de vértices pueden manifestarse como modos casi nulos. En el toy **equilátero** colapsamos intencionalmente la métrica a combinatoria, por lo que el “gauge” es mayormente **automorfismo combinatorio**, no difeomorfismo continuo.

Un siguiente paso hacia encodings de longitudes de Regge (aún finitos) usaría una paleta discreta de longitudes y tablas de tipos de triángulo; la fijación de gauge podría implementarse fijando un subconjunto de longitudes o agregando restricciones que seleccionen representantes canónicos. Se deja como trabajo futuro (§8).

7.3. Conexión al continuo (qué se requeriría)

Para conectar invariantes discretos verificados por SAT con expectativas del continuo, se requeriría como mínimo:

- secuencias de refinamiento (aumentando N o mallas por capas),
- paletas métricas crecientes (aproximando longitudes continuas),
- observables estables (distribuciones de curvatura, correladores) con tests de convergencia.

Nada en este artículo reemplaza ese programa; solo proveemos una base finita y auditable sobre la cual dicho programa puede construirse.

8. Limitaciones y extensiones futuras

- **Dureza:** las familias aquí son modestas. La síntesis de triangulaciones mayores se vuelve rápidamente desafiante sin encodings fuertes (redes de cardinalidad, PB constraints, manejo de simetrías) y solvers industriales.
- **MaxSAT / tolerancias:** restricciones físicas suelen requerir error acotado; MaxSAT u optimización pseudo-Boolean son extensiones naturales.
- **Dinámica cuántica Hamiltoniana:** el modelo booleano captura crecimiento del soporte causal, no amplitudes ni normas de conmutadores. Conectar con desigualdades LR completas requiere normas discretizadas y restricciones aritméticas.
- **3D/4D:** extender la síntesis de variedad a tetraedros/4-símplices es directo en principio, pero rápidamente exige mejores encodings y control de simetrías.

9. Software y disponibilidad de datos

El repositorio contiene:

- `gcnf_to_cnf.py`: reducción one-hot a DIMACS + mapa,
- `validate_solution.py`: validación del testigo contra GCNF original,
- `convert_solution.py`: mapeo entre CNF y asignaciones de dominio,
- `hess_cdcl.py`: solver prototipo tipo CDCL para GCNF.

Todas las instancias toy pueden generarse determinísticamente por los scripts descritos en el Apéndice B, y validarse con `validate_solution.py`.

A. Toolchain (comandos mínimos)

```
# 1) Convertir GCNF -> DIMACS CNF (+ map)
python gcnf_to_cnf.py instance.gcnf instance.cnf --map instance.map

# 2) Resolver CNF con tu solver SAT preferido (placeholder)
# cadical instance.cnf > sat.out

# 3) Convertir testigo CNF a asignaci\on de dominio (si se requiere)
python convert_solution.py --map instance.map --in sat.out --out instance.sol

# 4) Validar la asignaci\on contra el GCNF original
python validate_solution.py --gcnf instance.gcnf --sol instance.sol
```

B. Generadores de instancias (pseudocódigo)

B.1. Síntesis de triangulación (Toy-GR-Complex)


```
# Variables: tau_{ijk} para cada triple (i<j<k)
# Restricciones:
# (1) Para cada arista {i,j}: cantidad de tri\angulos incidentes en S_ij es 0 o 2
# (2) Para cada v\ertice v: suma de tau_{vij} igual al grado objetivo d_v (encoding de
    cardinalidad)
# Opcional: ruptura de simetr\{i}a (fijar un tri\angulo), usar con cuidado.
```

B.2. Dinámica booleana de localidad (Toy-LR-Dynamics)

```
# Variables: I(v,t) para v en V, t=0..T
# Base: I(v0,0)=1 e I(v!=v0,0)=0
# Update: I(v,t+1) <-> (I(v,t) OR OR_{u in N(v)} I(u,t))
# Query: I(target,T)=1
```

C. Ejemplo de cláusula GCNF

Una cláusula típica es una lista de literales de igualdad:

```
[(17, 0), (23, 1), (55, 1)]
```

interpretada como $(x_{17} = 0) \vee (x_{23} = 1) \vee (x_{55} = 1)$.

Referencias

- [1] T. Regge, General relativity without coordinates, *Il Nuovo Cimento* **19** (1961) 558–571. doi:[10.1007/BF02733251](https://doi.org/10.1007/BF02733251).
- [2] E. H. Lieb and D. W. Robinson, The finite group velocity of quantum spin systems, *Commun. Math. Phys.* **28** (1972) 251–257. doi:[10.1007/BF01645779](https://doi.org/10.1007/BF01645779).
- [3] B. Nachtergaele and R. Sims, Lieb–Robinson bounds and the exponential clustering theorem, *Commun. Math. Phys.* **265** (2006) 119–130. doi:[10.1007/s00220-006-1556-1](https://doi.org/10.1007/s00220-006-1556-1).
- [4] C. Sinz, Towards an optimal CNF encoding of Boolean cardinality constraints, in *CP 2005* (2005).
- [5] J. Ambjørn, A. Görlich, J. Jurkiewicz, and R. Loll, Nonperturbative quantum gravity, *Physics Reports* **519** (2012) 127–210. doi:[10.1016/S0370-1573\(12\)00301-8](https://doi.org/10.1016/S0370-1573(12)00301-8).
- [6] A. P. Gentle, Regge calculus: a unique tool for numerical relativity, *General Relativity and Gravitation* **34** (2002) 1701–1718.
- [7] S. H. Christiansen, On the linearization of Regge calculus, *Numerische Mathematik* **119** (2011) 613–640. doi:[10.1007/s00211-011-0394-z](https://doi.org/10.1007/s00211-011-0394-z).
- [8] Y. Feldman *et al.*, A systematic comparison of SAT encodings of linear constraints, in *CP 2024* (2024). (optional background)
- [9] M. Seidl *et al.*, pblib: A library for encoding pseudo-Boolean constraints into CNF, *SAT 2012 / later distributions*. (optional background)